

Laboranweisung 7: Automatisierung mit GitHub Actions

Bisher haben wir unseren Code nur lokal auf dem eigenen Rechner gebaut und manuell überprüft. Heute heben wir das Projekt auf Industrie-Standard: Wir verbinden unser lokales Git-Repository mit der Cloud (GitHub) und konfigurieren einen automatisierten CI-Workflow (Continuous Integration). Zukünftig wird jeder Code-Push von einem unbestechlichen Server-Roboter neu kompiliert und gegen eine harte Testsuite geprüft – vollautomatisch bei jedem Hochladen.

Vorbereitung: GitHub Account & Git Remote Setup

Bevor wir Code hochladen können, benötigen wir ein Ziel in der Cloud. Falls Sie letzte Woche bereits ein Remote eingerichtet haben, beachten Sie die Hinweise zur Umleitung.

1. GitHub Account Setup (Schritt-für-Schritt)

1. Öffnen Sie den Browser unter github.com und klicken Sie oben rechts auf "Sign up".
2. Geben Sie Ihre E-Mail-Adresse ein und erstellen Sie ein sicheres Passwort (mindestens 15 Zeichen oder starke Sonderzeichen-Kombination).
3. Wählen Sie einen passenden Usernamen.
4. Beantworten Sie die Frage nach Marketing-Mails mit "n" und lösen Sie das visuelle Captcha-Rätsel.
5. Öffnen Sie Ihr E-Mail-Postfach, kopieren Sie den Aktivierungscode und überspringen Sie das optionale Questionnaire ("Skip"). Sie befinden sich nun auf Ihrem Dashboard.
6. Wichtig: Git lokal auf Ihrem PC konfigurieren (falls noch nie geschehen):
 - `git config --global user.name "Dein Name"`
 - `git config --global user.email "deine.email@xyz.de"`

2. Das Cloud-Repository erstellen

Das Repository auf GitHub muss zum Start KOMPLETT LEER sein. Wenn GitHub automatisch eine README oder .gitignore generiert, unterscheidet sich der Cloud-Zustand von Ihrem lokalen Zustand. Die Folge beim ersten Push wäre ein blockierender Merge-Konflikt!

- Checkliste bei "Create a new repository":
 - Repository name: DigitalSimulator
 - Description: C++ Logic Gate Simulator with CI/CD

- Visibility: Public oder Private (für die Pipeline egal).
- NICHT ANKREUZEN: Add a README file 🖱️ Kritisch!
- NICHT ANKREUZEN: Add .gitignore
- NICHT ANKREUZEN: Choose a license
- Klicken Sie auf den grünen Button "Create repository".

3. Die Brücke in die Cloud bauen & Push

Nach dem Klick zeigt Ihnen GitHub die passenden Terminal-Befehle an. Öffnen Sie Ihr Terminal im Simulator-Projektverzeichnis auf Ihrem Laptop.

- Achtung für Codeberg-Nutzer der letzten Woche: Ihr lokaler Ordner hat bereits eine Verbindung namens origin. Der Befehl remote add wird mit fatal: remote origin already exists abbrechen. Nutzen Sie stattdessen den Befehl zum Umleiten:

```
git remote set-url origin https://github.com/IHR_NAME/DigitalSimulator.git
```

- Für alle anderen (Neues Remote anlegen):

```
git remote add origin https://github.com/IHR_NAME/DigitalSimulator.git
```

- Der initiale Push:

```
git branch -M main
```

```
git push -u origin main
```

(Hinweis: Windows-Nutzer werden beim ersten Push in einem Pop-up-Fenster vom Git Credential Manager nach dem GitHub-Login gefragt. Linux-Nutzer können ihre Credentials persistent mit `git config --global credential.helper store` sichern).

- Die Visitenkarte nachreichen: Legen Sie lokal im Hauptordner eine Datei namens README.md an, schreiben Sie # C++ Logik-Simulator hinein und pushen Sie diese kurz (`git add README.md`, `git commit -m "Füge README hinzu"`, `git push`), um die Startseite Ihres Repositories zu aktivieren.

Phase 1: Refactoring für den Roboter (Die Test-Suite)

Eine CI-Pipeline kann keine Konsolentexte wie `cout << "Gatter funktioniert"` lesen oder interpretieren. Die Cloud steuert Prozesse ausschließlich über numerische Exit-Codes des Betriebssystems. Liefert die `main()`-Funktion am Ende ein `return 0;`, gilt der Prozess als erfolgreich (Grüner Haken). Gibt das Programm `return 1;` (oder ein anderes `return != 0`) zurück oder stürzt durch einen Speicherfehler (Segmentation Fault) ab, schlägt die Pipeline fehl (Rotes Kreuz).

Bauen Sie Ihre main.cpp so um, dass für JEDES implementierte Logikgatter eine vollständige, unbestechliche Wahrheitstabelle geprüft wird.

- Für 2-Input-Gatter (AND, OR, XOR, NAND): Alle 4 Kombinationen (0-0, 0-1, 1-0, 1-1).
- Für 1-Input-Gatter (NOT): Beide Kombinationen (0, 1).
- Insgesamt: Mindestens 18 mathematische Testfälle. Bei jedem Fehler müssen die falschen Werte ausgegeben und das Programm hart abgebrochen werden.

Strukturvorgabe für die main.cpp:

```
#include <iostream>
// Hier Ihre Gatter-Header inkludieren (z.B. #include "AndGate.h")

int main() {
    // 1. Globaler Test-Status
    bool testPassed = true;

    std::cout << "--- STARTE AUTOMATISIERTE WAHRHEITSTABELLEN-TESTS ---" <<
std::endl;

    // 2. Beispiel-Testblock für ein AND-Gatter (Stellvertretend für alle
Gatter)
    {
        // Instanziierung (Nutzen Sie Ihre Smart-Pointer-Architektur von
letzter Woche!)
        auto andGate = std::make_unique<AndGate>();

        // Test-Matrix: InputA, InputB, Erwartetes Ergebnis
        int testCases[4][3] = {
            {0, 0, 0},
            {0, 1, 0},
            {1, 0, 0},
            {1, 1, 1}
        };

        for (int i = 0; i < 4; ++i) {
            andGate->setInput(0, testCases[i][0]);
            andGate->setInput(1, testCases[i][1]);
            andGate->evaluate();

            if (andGate->getOutput() != testCases[i][2]) {
                std::cerr << "✗ TEST FAILED: AND bei Inputs A=" <<
testCases[i][0]
                    << " B=" << testCases[i][1]
                    << " -> Erhalten: " << andGate->getOutput()
                    << " (Erwartet: " << testCases[i][2] << ")" <<

```

```

std::endl;
        testPassed = false;
    }
}

// [!] HIER WEITERE TESTBLÖCKE FÜR OR, NOT, XOR, NAND ERGÄNZEN (Insgesamt
min. 18 Fälle)

// 3. Finale Auswertung für die CI-Pipeline
if (!testPassed) {
    std::cerr << "--- 🚫 PIPELINE-ABSTURZ: TESTS FEHLGESCHLAGEN ---" <<
std::endl;
    return 1; // Signalisiert GitHub Actions: FEHLER!
}

std::cout << "--- 🟢 ALLER TESTS BESTANDEN (18/18) ---" << std::endl;
return 0; // Signalisiert GitHub Actions: ERFOLG!
}

```

Kompilieren und testen Sie diese Logik lokal auf Ihrem PC, bevor Sie fortfahren.

Phase 2: Infrastructure as Code (Die YAML-Bauanleitung)

Wir konfigurieren nun das Verhalten des Cloud-Servers. GitHub Actions sucht nach Workflows ausschließlich im strikt vordefinierten Pfad `.github/workflows/`.

1. Erstellen Sie im Hauptverzeichnis Ihres Projekts einen Ordner namens `.github` (Achtung auf den führenden Punkt!).
2. Erstellen Sie innerhalb dieses Ordners einen Unterordner namens `workflows`.
3. Legen Sie im Ordner `workflows` eine neue Textdatei mit dem Namen `ci.yml` an.
4. Schreiben Sie folgenden Inhalt in die Datei. Achtung: YAML erlaubt keine Tabulatoren für die Einrückung. Nutzen Sie stattdessen Leerzeichen (2 Spaces pro Ebene)!

```

name: C++ Automated Test Suite

on:
  push:
    branches: [ "main" ] # Pipeline startet bei jedem Push auf den main-Branch

jobs:
  build-and-test:
    runs-on: ubuntu-latest # Startet eine komplett frische Linux-Maschine

```

(Ubuntu)

steps:

- name: Quellcode auschecken

uses: actions/checkout@v4 # Kopiert den C++ Code aus dem Repo auf den Runner

- name: Kompiliere den Simulator

Flags erklärt:

-std=c++17 zwingend nötig für Smart Pointer (make_unique)

-Wall -Wextra aktiviert alle wichtigen Compiler-Warnungen

run: g++ -std=c++17 -Wall -Wextra *.cpp -o simulator

- name: Führe Wahrheitstabellen-Tests aus

Weiche für Betriebssystem-Kompatibilität beim Ausführen des Binaries

run: |

if [-f "./simulator"]; then

./simulator

else

./simulator.exe

fi

Phase 3: Push, Live-Überwachung & Build Badge

1. Sichern und pushen Sie die Pipeline in die Cloud:

git add .

git commit -m "Füge GitHub Actions CI-Pipeline hinzu und refactor main.cpp Tests"

git push origin main

2. Wechseln Sie sofort in den Browser auf Ihre GitHub-Projektseite und klicken Sie auf den Reiter "Actions".
3. Klicken Sie auf den obersten, laufenden Workflow-Eintrag (gelber Kreis bedeutet: läuft gerade). Klicken Sie links auf den Job build-and-test, um die Terminalausgabe des Linux-Runners live einzusehen.

Das Status-Badge einbinden

Ein Badge zeigt den Gesundheitszustand Ihres Codes direkt dekorativ auf der Startseite an. Es ist völlig egal, ob Ihre Pipeline aktuell grün oder rot ist – das Badge aktualisiert sich bei jedem Push dynamisch von selbst.

1. Klicken Sie in der Actions-Oberfläche des Workflows oben rechts auf den Button mit den drei Punkten "... " und wählen Sie "Create status badge".

2. Belassen Sie die Einstellungen auf Branch: main und klicken Sie auf "Copy status badge Markdown".
3. Öffnen Sie lokal Ihre README.md und fügen Sie den kopierten Text (sieht aus wie ![C++ Automated Tests](../badge.svg)) in die allererste Zeile der Datei ein.
4. Committed und pushen Sie die Änderung. Auf der GitHub-Hauptseite prangt nun Ihr dynamisches Echtzeit-Zertifikat!

Zusatzaufgaben für Schnelle

Zusatzaufgabe 1: Statische Code-Analyse mit cppcheck

Man kann Code nicht nur kompilieren, sondern ihn auch automatisiert vor dem Build auf Programmierfehler, ungenutzte Variablen oder schlechten Stil scannen lassen (Linting). Dafür nutzen wir das Industrie-Tool cppcheck.

- Erweitern Sie Ihre ci.yml um die Installation und Ausführung von cppcheck direkt vor dem Compile-Schritt:

```
- name: Installiere cppcheck Statische Analyse
  run: sudo apt-get update && sudo apt-get install -y cppcheck

- name: Code-Qualität prüfen mit cppcheck
  # --error-exitcode=1 sorgt dafür, dass die Pipeline bei Fehlern stoppt
  run: cppcheck --enable=all --error-exitcode=1 *.cpp
  continue-on-error: true # Verhindert, dass das Labor abbricht, falls
  Stilfehler existieren
```

- Pushen Sie den Code und analysieren Sie das Ergebnis im Log. cppcheck kategorisiert Fehler genau:
 - KRITISCH (Sofort beheben!): Memory leak: comp (new ohne delete verwendet → Refactoring zu Smart Pointern nötig!) oder Null pointer dereference.
 - WARNUNG / STYLE (Sollte behoben werden): Variable assigned but not used oder Style: Function should have const.

Zusatzaufgabe 2: Ernstfall-Simulation & KI-Untersuchung (Lernziel: Fehlerdiagnostik)

Lernen Sie, wie man mit fehlschlagenden Pipelines professionell umgeht, ohne durch unüberlegtes "Copy-Paste" die Software-Architektur zu zerstören.

1. Sabotage: Öffnen Sie Ihre AndGate.cpp und ändern Sie die funktionierende Bit-Operation absichtlich von UND (&) auf ODER (|):

```
// Sabotage im Code:  
output = inputA | inputB; // FALSCH!
```

2. Push & Crash: Pushen Sie den fehlerhaften Code. Ihre Pipeline im GitHub-Actions-Tab wird nach kurzer Zeit fehlschlagen und ein rotes Kreuz anzeigen.
3. Log-Isolierung: Öffnen Sie den Log des fehlgeschlagenen Schritts "Führe Wahrheitstabellen-Tests aus". Scrollen Sie ganz nach unten und kopieren Sie NUR die Zeilen des fehlgeschlagenen Gatter-Tests, nicht den gesamten, unleserlichen 200-Zeilen-System-Log!

(Beispiel: ✗ TEST FAILED: AND bei Inputs A=1 B=1 -> Erhalten: 1 (Erwartet: 1))

4. Strategischer KI-Prompt: Füttern Sie eine KI (ChatGPT/Gemini) mit folgendem, restriktiven Prompt, um das C++ Problem tiefgreifend zu verstehen, anstatt sich den Code diktieren zu lassen:

"Ich programmiere einen C++ Digitalsimulator. Mein automatisierter Wahrheitstabellen-Test wirft in der CI-Pipeline folgenden Log aus: [KOPIERTEN FEHLER HIER EINFÜGEN]. Ich habe in meinem AndGate fälschlicherweise den Bitwise-Operator | statt & genutzt. Erkläre mir bitte in zwei Sätzen den rein konzeptionellen, mathematischen Unterschied dieser beiden Operatoren in C++, ohne mir eine fertige Code-Lösung zu generieren."

5. Reparatur: Korrigieren Sie den Fehler auf Basis der Erklärung eigenhändig in Ihrer IDE, committen Sie mit einer aussagekräftigen Nachricht (git commit -m "Fix: Corrected bitwise operation in AndGate") und bringen Sie Ihr Build-Badge zurück auf ein glänzendes, grünes passing!